



Especificación de Diseño de Software

Gonzalez Castillo Alexander de Jesus

Gonzalez Castillo Roberto de Jesus

Universidad Cenfotec

SOFT-11-C1 Proyecto Integrador 1 SCV0

Verónica Mora Lezcano

Julio, 2026

Índice

Índice.....	2
Propósito y alcance del sistema.....	3
Arquitectura.....	3
Diagrama de arquitectura:.....	5
Base de Datos.....	6
Tecnologías Utilizadas.....	7
Modelado del sistema:.....	7
Interfaz de usuario (UI/UX):.....	7
Prototipos:.....	8
Guia de estilos:.....	8
Restricciones de usabilidad y accesibilidad:.....	8
Diseño de navegación:.....	8

Propósito y alcance del sistema

Bajo la administración actual del festival estudiantil se presentan numerosos problemas derivados del uso de hojas de cálculo, formularios externos, mensajes de WhatsApp y registros manuales. El sistema CompusFest pretende centralizar y automatizar la gestión del festival con el fin de eliminar estos problemas. El sistema permitirá:

- Crear, modificar y eliminar actividades para el festival (culturales, deportivas, tecnológicas, artísticas, gastronómicas y recreativas)..
- Consultar las actividades del festival.
- Gestionar inscripciones de participantes con control de duplicados, cupos y lista de espera.
- Administrar stands o grupos participantes.
- Visualizar una agenda cronológica actualizada con el estado de cada actividad (disponible, lleno o cancelado).
- Contactar a la administración del festival.
- Proveer una experiencia de usuario consistente tanto en escritorio como en dispositivos móviles, incluyendo modo oscuro automático según la preferencia del sistema operativo.

El sistema se desarrollará en dos etapas: una primera centrada en la interfaz visual (frontend estático) y una segunda que incorporará la lógica de negocio y persistencia de datos.

Arquitectura

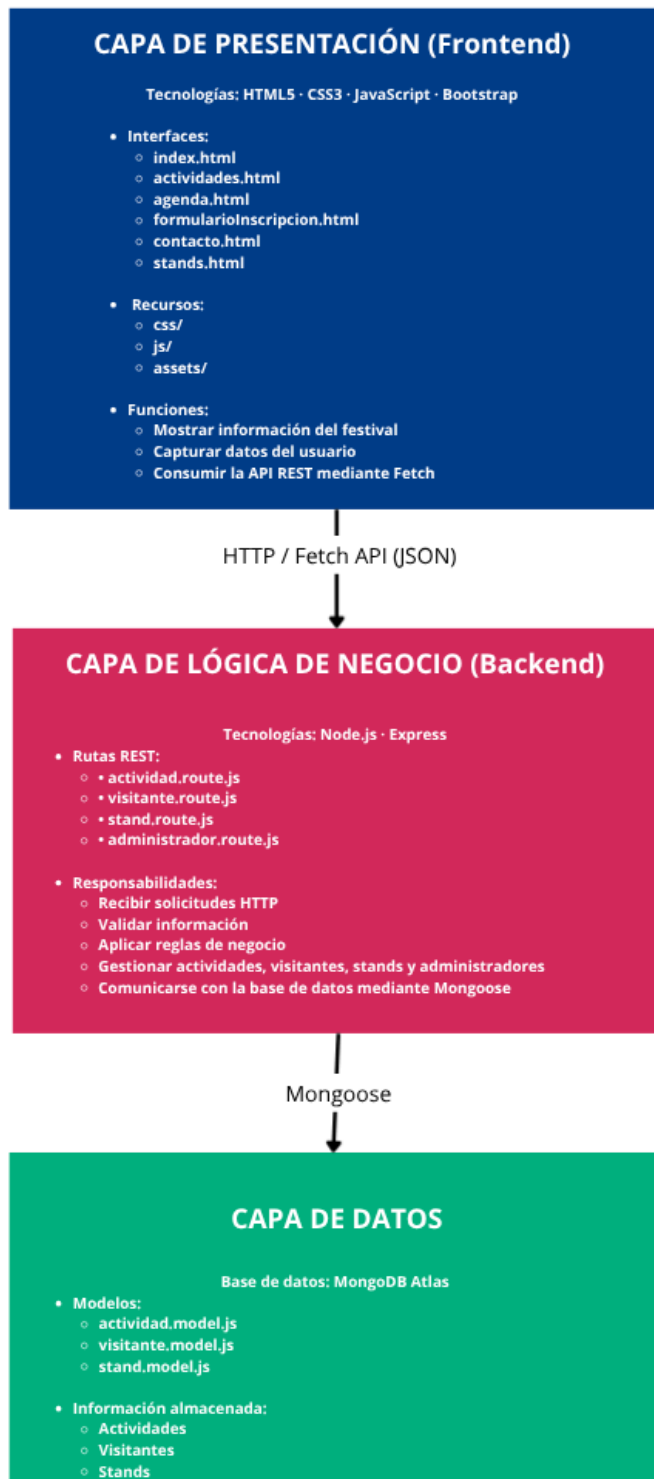
Se implementará una arquitectura cliente-servidor de tres capas, siguiendo el patrón Modelo-Vista-Controlador (MVC) para asegurar la separación de responsabilidades:

- Capa de presentación (Vista): Interfaces HTML5, CSS y JavaScript del lado del cliente, construidas con el framework Bootstrap para garantizar un diseño responsivo y accesible. Las vistas incluirán las páginas de inicio, catálogo de actividades, detalle de actividad, formulario de inscripción, agenda, stands, contacto, así como las pantallas de administración (gestión de actividades, stands, participantes y resultados).

- Capa de controlador (Controlador): Implementada en Node.js, gestionará las peticiones HTTP, orquestrará las validaciones, aplicará las reglas de negocio (inscripción 36 horas antes del evento, control de cupos, lista de espera, entre otros) y comunicará los modelos con las vistas.
- Capa de datos (Modelo): Gestionará la persistencia utilizando MongoDB Atlas como base de datos NoSQL en la nube. Los modelos representarán entidades como Actividad, Participante, Inscripción, Stand, Administrador y ListaEspera.

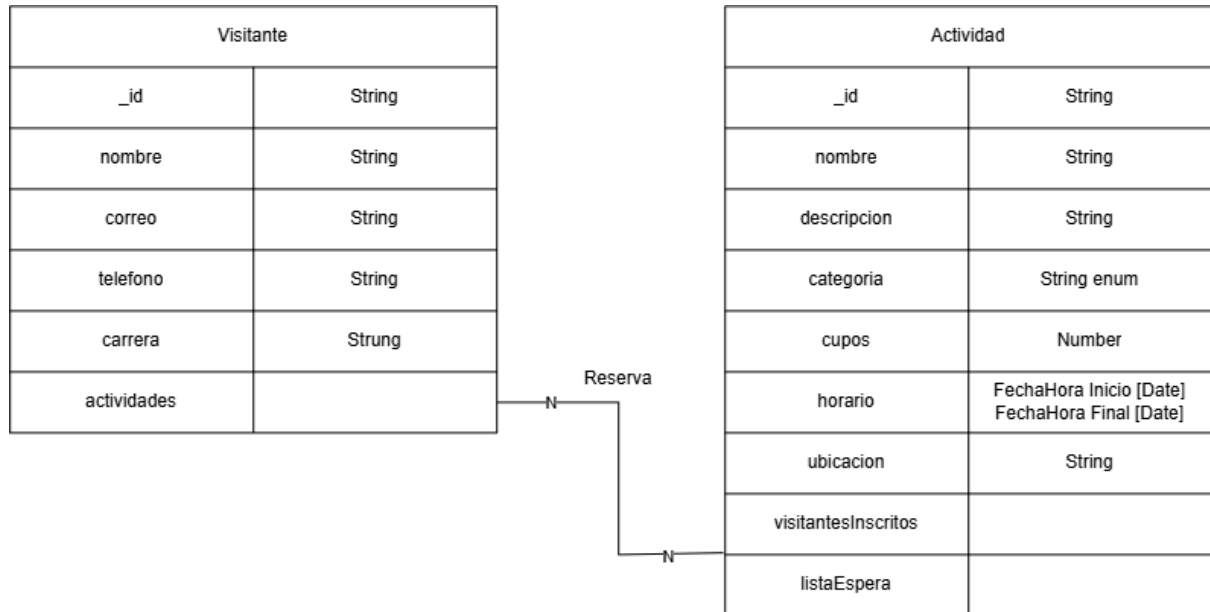
La comunicación entre el cliente y el servidor se realizará mediante peticiones Fetch a una API RESTful que retornará datos en formato JSON, permitiendo una experiencia dinámica sin recargas completas de página.

Diagrama de arquitectura:



Base de Datos

Se utilizará MongoDB Atlas por su flexibilidad para almacenar documentos con estructura variable (por ejemplo, actividades con diferentes requisitos, listas de espera embebidas) y por su escalabilidad en la nube.



Stands	
_id	String
nombre	String
categoria	String enum
responsable	String
ubicacion	String
descripcion	String

Tecnologías Utilizadas

Capa	Tecnologías
Frontend	HTML5, CSS, JavaScript, Bootstrap
Backend	Node.js, Express.js
Base de datos	MongoDB Atlas
Entorno de desarrollo	Visual Studio Code
Pruebas	Navegadores (Chrome, ...), Extensión de Live Server en Visual Studio Code
Control de Versiones	Git
Repositorio	GitHub

Modelado del sistema:

<https://github.com/verbrannt-bob/SOFT-11-CampusFest/blob/main/documentacion/diagramas/casos-de-uso.pdf>

Interfaz de usuario (UI/UX):

Prototipos:

<https://github.com/verbrannt-bob/SOFT-11-CampusFest/tree/main/documentacion/UI>

Guia de estilos:

Se utilizaran los colores, tipografía, iconos y logos definidos por Libro de Marca de la Universidad Cenfotec.

<https://github.com/verbrannt-bob/SOFT-11-CampusFest/blob/main/documentacion/UI/libro-marca-cenfotec.pdf>

Restricciones de usabilidad y accesibilidad:

La interfaz deberá ser consistente, con controles claramente identificables, navegación por teclado y contraste mínimo suficiente entre texto y fondo para facilitar su uso por personas con distintas capacidades. Además, no se permitirá depender únicamente del color para comunicar información importante, y todas las imágenes e iconos funcionales deberán incluir texto alternativo. Estas pautas están alineadas con principios básicos de accesibilidad y usabilidad para que el sistema pueda ser comprendido, aprendido y usado por la mayor cantidad de usuarios posible.

Diseño de navegación:

<https://github.com/verbrannt-bob/SOFT-11-CampusFest/blob/main/documentacion/diagramas/mapa-de-sitio.pdf>

Estructura de Sistema:

```
CampusFest/
├── backend/
│   ├── models/
│   ├── node_modules/
│   └── routes/
└── frontend/
    ├── assets/
    ├── css/
    └── js/
```


API REST:

Visitantes:

Ruta: `"/visitantes"`

- Ruta: `"/"`; Método: `POST`
 - Crear nueva inscripción en la colección visitantes
- Ruta: `"/"`; Método: `GET`
 - Solicitar todos los registros de la colección visitantes
- Ruta `"actividades/:id"`; Método: `PATCH`
 - Modificar el array actividades

Actividades:

Ruta: `"/actividades"`

- Ruta: `"/"`; Método: `POST`
 - Crear nueva actividad en la colección actividades
- Ruta: `"/"`; Método: `GET`
 - Solicitar todos los registros de la colección actividades
- Ruta: `"/:id"`; Método: `GET`
 - Solicitar un registro específico por `_id` de la colección actividades
- Ruta: `"/:id"`; Método: `PUT`
 - Modifica un registro específico por `_id` de la colección actividades
- Ruta: `"/:id"`; Método: `DELETE`
 - Eliminar un registro específico por `_id` de la colección actividades
- Ruta `"visitantes/:id"`; Método: `PATCH`
 - Agregar un visitante al campo de visitantesInscritos
- Ruta `"espera/:id"`; Método: `PATCH`
 - Agregar un visitante a la listaEspera
- Ruta `"cupos/:id/:cupos"`; Método: `PATCH`
 - Modificar el cupo máximo de una actividad en específico
- Ruta `"admitir/:id"`; Método: `PATCH`
 - Mover cantidad específica de listaEspera a visitantesInscritos

Stands:

Ruta: `"/stands"`

- Ruta: `"/"`; Método: `POST`
 - Crear nuevo Stand en la colección stands
- Ruta: `"/"`; Método: `GET`
 - Solicitar todos los registros de la colección stands
- Ruta `"/:id"`; Método: `GET`
 - Solicitar un registro específico por `_id` de la colección stands
- Ruta `"/:id"`; Método: `PUT`
 - Modificar un registro específico por `_id` de la colección stands
- Ruta `"/:id"`; Método: `DELETE`
 - Eliminar un registro específico por `_id` de la colección stands

Seguridad:

- Los métodos de API REST se diseñan con una estructura de try catch para manejar errores de conexión, u otros posibles errores imprevistos en los métodos.
- Además tanto en los formularios desde el frontend como en los mismos métodos REST se validan que estén todos los datos requeridos presentes y cuando es el caso, que tengan el formato correcto.